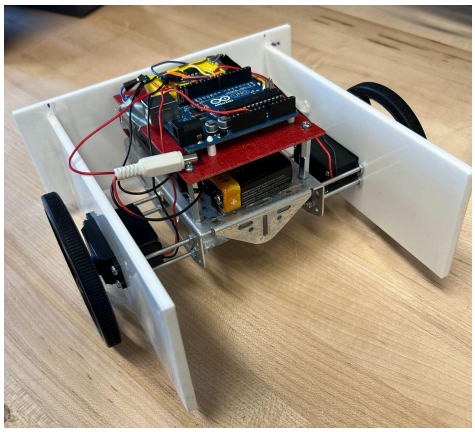
Final Report

Group 5

Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

Robot Design and Strategy Overview



The mechanical design of our robot focuses on optimizing the internal space available for collecting cubes. The chassis is elevated by larger wheels that sit on the outside of a laser-cut white acrylic perimeter, as shown on the right. The perimeter forms three sides of a roughly 8 x 8 inch square. The usable cube-collecting area is slightly smaller than the maximum 8 x 8 inch limit to accommodate the width of the wheels and a reserved section in the back of the chassis for the color sensor. However, the design still provides sufficient space to collect blocks.

The robot's electrical system consists of two L9110 H-bridges, two DC wheel motors, an Arduino Uno, and a color sensor. Each H-bridge controls one DC motor. The output pins of the H-bridges are connected to the positive and negative terminals of the motors, while the input pins are connected to the digital output pins on the Arduino. By changing the input signals, the Arduino controls the direction of the wheel rotation, and thus the overall robot motion. The robot also uses a color sensor connected as an input to the Arduino. The H-bridges are powered by a 6V (4 AA batteries in series), while the Arduino is powered by a 9V battery.

The software design is based on open-loop control and consists of motor functions as well as an interrupt based on the color sensor's reading. The motor functions include drive_forward(), turn_left(), and stop(), which operate by setting the appropriate H-bridge input pins to control the relative movement of the wheels. The robot uses a timer interrupt to measure the frequency output of the color sensor. The robot stores the detected period corresponding to the initial surface color, and then turns when the measured period changes by more than a calibrated difference threshold of 100, which indicates a switch from blue to yellow or vice-versa. Additionally, if the period exceeds a calibrated absolute threshold corresponding to the color black, the robot stops.

During competition, the robot is set at a roughly 30 degree angle from the back black border, facing rightward. It deploys after plugging in the 9V battery to the Arduino. Then, ideally, the robot drives forward, detects the blue-yellow border and turns left. The degree of the turn is pre-calibrated by adjusting the delay after the turn_left() function such that the robot is oriented parallel to the blue-yellow border. Then, it drives forward, collecting blocks until it reaches the black border and stops. In

practice, this was not achieved due to friction from the blocks. Since our software is open-loop, it does not allow for direction correction.

Design Process Reflection

We started the design process by meeting and discussing ideas we had for the overall robot functionality. We used drawings to communicate different ideas. These were done by hand and then adjusted to fit the specifications in the assignment using CAD. We also decided to orient the robot backwards to give us more room to gather blocks. We approached the milestones by completing the second with just the barebones chassis, during which we faced our first challenge. One of our motors had a wire come loose, and our replacement motor fit poorly with our wheel. This was a standing challenge that we had to deal with as it caused the robot to not drive straight.

Our first robot was made using cardboard before using more expensive materials like acrylic. It also allowed us to decide if we wanted to include certain design choices we were considering at first. The primary change we ended up making was not including a gate mechanism to hold blocks that we captured. We realized that the amount of work it would take in addition to space constraints made it difficult to follow through with this, and it didn't make a significant impact on the number of blocks gathered.

We then manufactured our acrylic sheets for the side walls of the robot and adjusted the wall for the back as we knew, with the ball wheel, the spacing would be difficult to cut with acrylic accurately without making it drag on the ground or letting blocks through. We had to play around with the placement of the back wall to avoid these issues. The main roadblocks we ran into for the remaining milestones were just with our color sensor not differentiating black and blue as well as it had initially due to the positioning on our newly completed robot. We ended up blocking the colors of the blocks from interfering with the sensor with some cardboard, which worked well in our final competition. Since we didn't use two color sensors, we were unable to program more complex continuous motion for our robot, meaning we only detected when the robot moved to the other side of the board, made it turn to capture blocks, and stopped it when it hit the border. We had to account for the natural tendency of the robot to turn because of the faulty wheel while programming the motion, but all-in-all, it was able to perform well with some slight adjustments.

Competition Analysis

Our robot performed very well in the tournament. The color sensing worked well and we never seemed to have a situation where it did anything we didn't anticipate. While the motion we coded worked well for the most part, some situations where our robot got entangled with another robot caused us to wonder if we should have included another sensor to detect the other robot and react accordingly. There were also a couple of rounds that our robot got pushed off the board. This is once again something that might have been avoided if we had included an ultrasonic sensor for the other robot and accounted for when a robot is approaching. We were honestly

surprised at how many blocks we were able to gather. We felt our robot took more time than some others to start its motion and realized speed was of the essence, particularly in the more successful robots we observed. Ultimately, our robot was robust and reliable, winning us most of the rounds. The areas we could have improved involved other improvements and complexities that we felt, while designing and even after the fact, were a risk that may have impaired the overall functionality of our robot rather than helping it.

Conclusions

We believe our robot to be successful, as we met all the core requirements and collected a significant amount of cubes. However, as we watched our robot compete, we recognized how we could improve our sensor selection and logic.

If we were to redo this project, one of the first changes we would make is implementing a “turn and move away from the black border” response rather than a “stop on the black border” response. Our code made it so that if our robot detected the black border after it had already performed one turn, it would stop. Although this made our code simple to implement, it prevented our robot from maximizing cube collection. There were rounds where our robot detected the black border very early in the match due to being pushed by the opposing robot. This resulted in our robot being stationary for the majority of the round. We would edit our code so that the robot recognizes the black border, reverses, and turns away from the boundary into the middle of the board to resume collecting cubes. This change would allow us to maximize the number of cubes collected.

Another change we would make is to remove the delay at the beginning of our code. This delay caused our robot to take a few seconds to move after the match had started. Throughout each round, we realized that speed is a significant advantage. There were matches where we were unable to collect cubes due to the opposing robot blocking our path. This change would allow us to access cubes and approach an optimal path before the opposing robot got in the way.

Our final and most complex change would be to implement QTI sensors into our robot. This would allow our robot to detect not only that it had encountered the border, but specifically which side our robot was contacting it. Making this change would give our robot an added directional awareness that would allow it to always turn back into the center of the playing area. Our lack of QTI sensors is what led us to make the decision to simply stop when it detected black and to start each round in the exact same orientation. QTI sensors would allow us to start our robot in any orientation, giving us a spatial advantage.

For students doing this project next year, we would advise them to be bold and not play it safe. We played it safe with a simple design and simple logic. Although our robot was successful, we believe that we would have made it further in the competition if we had taken chances on more complex implementations. A robot that adapts and reacts is always going to dominate over a robot that pauses, hesitates, or stops.

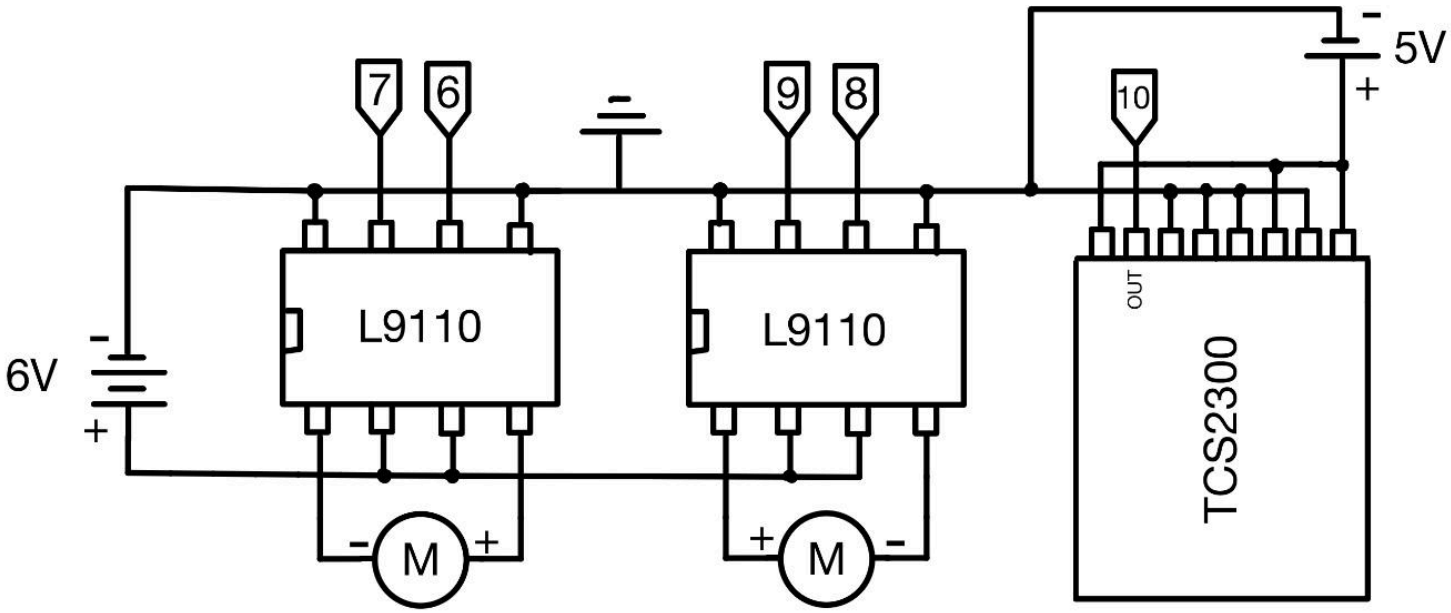
Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

Appendix A: Bill of Materials

Item Name	Unit Price	Quantity	Item ID	Vendor	Additional Manufacturing Cost	Total Cost
Wheels	$2	2	N/A	From lab	N/A	$4
Structural Adhesive, Waterproof Epoxy, J-B Weld Clearweld, 0.5 FL.oz Tube	$7.11	1	7605A25	McMaster	N/A	$7.11
Steel Pan Head Phillips Screw, M3 x 0.5 mm Thread, 50 mm Long	$7.32	1	92005A137	McMaster	N/A	$7.32
Acrylic Sheet	$5	1	N/A	RPL	0.5*3 + 0.05*64	$4.70
TOTAL COST						**= 23.13**

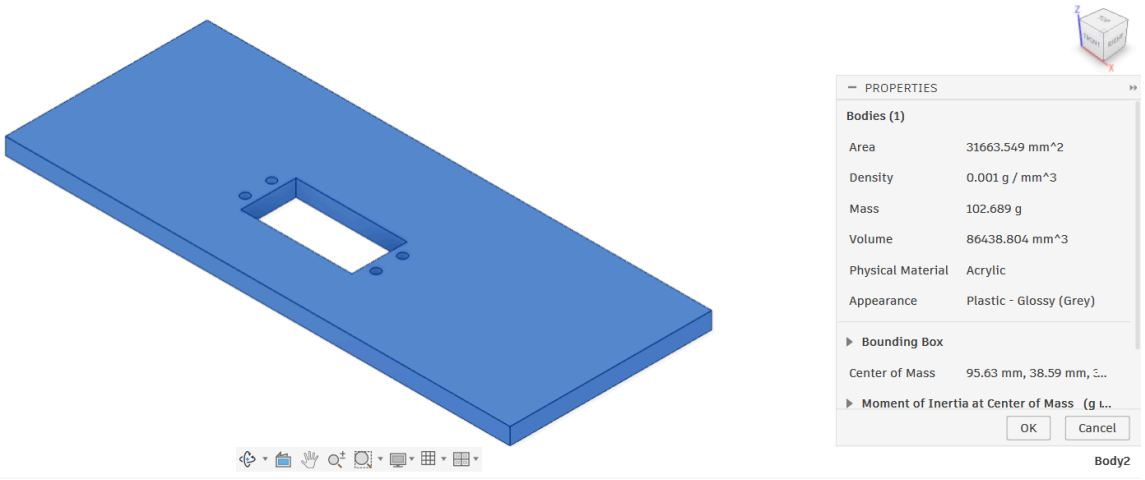
Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

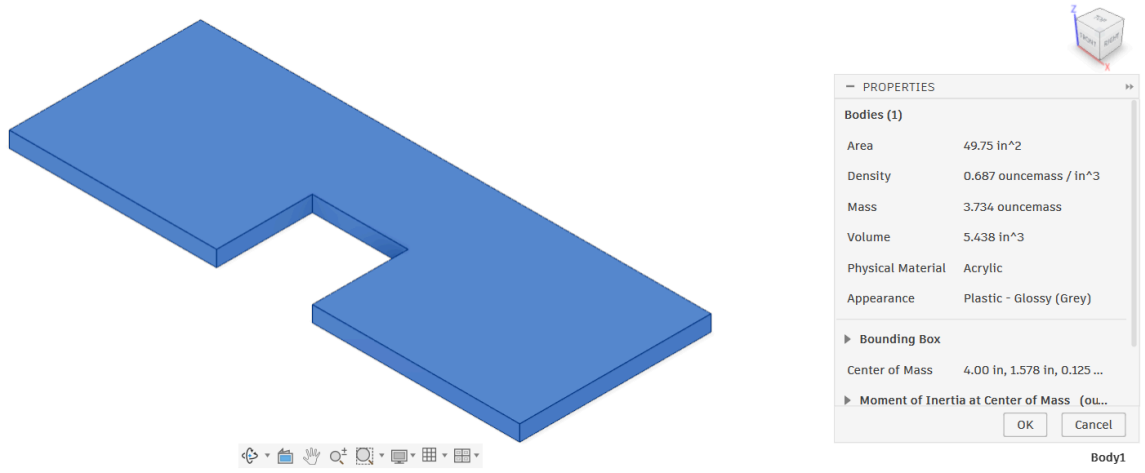
Appendix B: Circuit Diagram



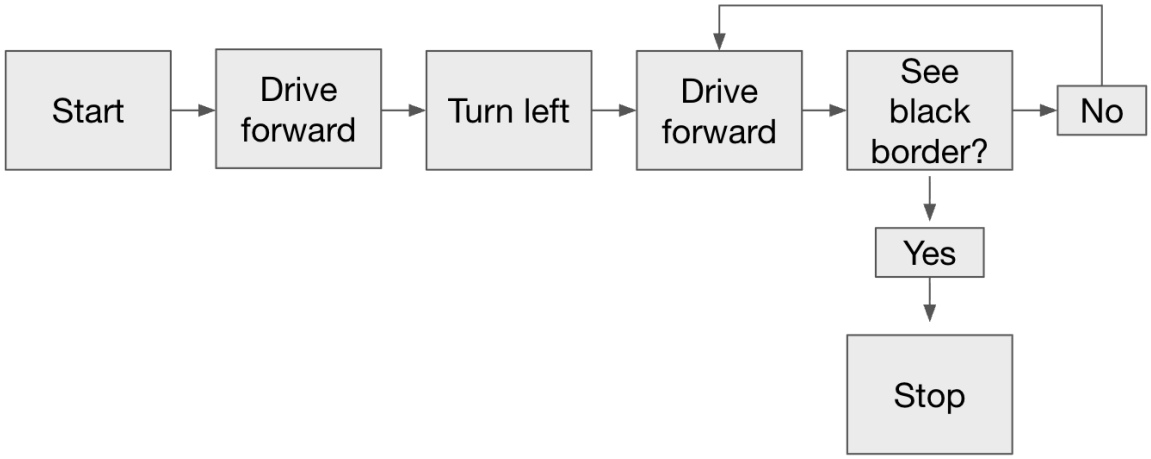
Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

Appendix C: CAD Files and Drawings





Appendix D: Flowchart



Rebecca Gerola (rdg245), Shreya Anand (sa2228), and Kayoko Thornton (ket65)

Appendix E: Code

```
// Global Variables
volatile unsigned int timerVal;
int startPeriod;
int blackThreshold = 300; // Calibrated for Black
int colorDiffThreshold = 100; // Difference between Blue and Yellow

// ISR: Tracks the signal from the color sensor on Pin 10 (PB2)
ISR(PCINT0_vect) {
  // Check Pin 10 (bit 2 of Port B)
  if (PINB & 0b00000100) {
    TCNT1 = 0;  // Reset timer on rising edge
  } else {
    timerVal = TCNT1; // Capture counts on falling edge
  }
}

// initColor: Setup for Pin 10 and Timer 1. Referenced from pre-lab 4
void initColor() {
  // Set Pin 10 (PB2) as input
  DDRB &= ~0b00000100;

  // Enable Pin Change Interrupt 0 (covers PB0-PB7)
  PCICR |= 0b00000001;

  // Timer 1: Normal mode, Prescaler 1
  TCCR1A = 0;
  TCCR1B = 0b00000001;
  sei(); // Enable global interrupts
}

// getColor: Samples the sensor frequency and returns the period
int getColor() {
  PCMSK0 |= 0b00000100; // Enable interrupt for Pin 10
  _delay_ms(15);        // Sampling window
  PCMSK0 &= ~0b00000100; // Disable interrupt
  return (2 * timerVal / 16);
}

// Motor Functions
void drive_forward() {
```

```
  PORTB = (PORTB & ~0b00000001) | 0b00000010; // Pin 8 Low, Pin 9 High
  PORTD = (PORTD & ~0b01000000) | 0b10000000; // Pin 6 Low, Pin 7 High
}

void turn_left() {
  PORTB = (PORTB & ~0b00000010) | 0b00000001; // Pin 9 Low, Pin 8 High
  PORTD = (PORTD & ~0b01000000) | 0b10000000; // Pin 6 Low, Pin 7 High
}

void stop() {
  PORTB &= ~0b00000011; // Clear pins 8 & 9
  PORTD &= ~0b11000000; // Clear pins 6 & 7
}


// Competition Algorithm
int main(void) {
  Serial.begin(9600);
   DDRB |= 0b00000011; // set pins 8 and 9 as outputs for "left" motor
   DDRD |= 0b11000000;  // set pins 6 and 7 as outputs for "right" motor
  initColor();

  // Detect Initial Color
  _delay_ms(1000);
  startPeriod = getColor();

  // Drive until the other color is reached
  drive_forward();
  while (1) {
  int current = getColor();

  if (abs(current - startPeriod) > colorDiffThreshold && current < blackThreshold) {
    stop(); //check again to make sure
    if (abs(current - startPeriod) > colorDiffThreshold && current < blackThreshold) {
      break;
      }
  }
}
  // turn
  _delay_ms(100);
  turn_left();
  _delay_ms(500);
```

```
  stop();

  // drive until Black Edge
  _delay_ms(600);
  drive_forward();
  while(1) {
    if (getColor() > blackThreshold) {
      stop(); // Stop immediately at the edge
      break;
    }
  }
  while(1);
//end
}
```